

Assignment: Kanban with Comments & @Mentions (Track 1: Beginner)

Claude Code Training — Track 1

April 24, 2026

Overview

In this assignment you will build a small but complete Kanban board application with comment threads and @mention notifications, starting from one of three provided starter repos (Laravel, FastAPI, or Next.js). The feature set is intentionally modest — the point of the exercise is not to ship a novel product, but to practice a disciplined workflow with Claude Code end to end.

You will practice the following workflow skills: writing user-level and project-level CLAUDE.md files (both by hand and via `/init`), running an explicit design phase before touching code, delegating planning to a subagent, producing an architecture document with a subagent or plugin, implementing the feature test-first, and finally packaging the work into a self-contained HTML workflow report that another engineer could review without asking you a single question.

Time Budget

Total time: **3-4 hours**.

Phase	Time
Setup + starter repo	20 min
CLAUDE.md files (manual + <code>/init</code>)	30 min
Design phase	30 min
Plan phase (subagent)	30 min
Architecture phase	30 min
TDD implementation	90 min
HTML workflow report	20 min

If you find yourself over budget, cut scope on the UI polish before you cut scope on the workflow artifacts. The workflow is what is being graded.

What You Will Build

A single-user Kanban board where a signed-in user can organize cards across columns, comment on cards, and receive notifications when someone (in the beginner track, that someone is themselves in a second browser or a seeded test user) @-mentions them.

Boards

- A user can create, rename, and delete a Board.
- Boards belong to a single user (no sharing yet).
- **Acceptance:** `/boards` shows a list of my boards; `/boards/new` creates one; `/boards/{id}` shows one board with its columns and cards.

Columns

- Each Board has one or more Columns. Default seed on board creation: "Todo", "Doing", "Done".
- User can add, rename, reorder, and delete columns within a board.
- **Acceptance:** reordering persists across a page reload.

Cards

- Each Card lives in a Column and has: title (required), description, assignee (optional; self only in the beginner track).
- User can create a card in any column, edit it, delete it, and drag it between columns.
- **Acceptance:** dragging a card to a new column updates its column association in the database, not only in the client.

Comments

- Each Card supports threaded comments (one level deep is fine — no nested replies).
- Comment body supports @mentions of the form `@username`.

@Mentions

- The comment renderer parses `@username` tokens, links each mention to that user's profile page (a placeholder route is acceptable), and creates a Notification row for the mentioned user.
- The mentioned user sees an unread mention in a notifications feed at `/notifications`.
- Marking a notification read happens on click.

Notifications feed

- `/notifications` shows a chronological list of the current user's notifications, unread first.
- An unread count appears in the nav bar on every page.

Choose Your Starter Repo

Stack	Path	Test command
Laravel	<code>starter-repos/laravel-starter/</code>	<code>php artisan test</code>
FastAPI	<code>starter-repos/python-fastapi-starter/</code>	<code>pytest</code>
Next.js	<code>starter-repos/nextjs-starter/</code>	<code>npm test</code>

Follow the starter repo's README first — get it running locally before you touch anything else. If the tests don't pass on a fresh clone, fix that before starting the assignment.

Workflow (Six Steps)

The workflow below is deliberately explicit. Every phase produces an artifact you commit to the repo. The artifacts are what gets graded, so treat them as first-class deliverables.

Step 1 — CLAUDE.md files

Claude Code reads two CLAUDE.md files on startup: a user-level file at `~/ .claude/CLAUDE.md` that applies to every project, and a project-level file at the repo root that applies only to this codebase.

- **User CLAUDE.md** (`~/ .claude/CLAUDE.md`) captures global preferences: your language and style conventions, testing philosophy, comment density, how you like Claude to communicate with you.
- **Project CLAUDE.md** (repo root) captures project-specific information: the stack, folder layout, key files, test and run commands, and the two or three quirks a fresh model would never guess.

Deliverable: submit BOTH a manual version and a `/init`-generated version of each file. Commit them under `docs/claude-md-manual/` and `docs/claude-md-init/`. Add a short `docs/claude-md-notes.md` reflecting on the differences.

Tip. Don't blindly accept `/init` output. Read every line, delete boilerplate that doesn't apply to your project, and add the two or three project quirks the model would never guess.

Step 2 — Design phase

Use Claude Code to help sketch the UI before you write a line of application code. Do this in a chat session, not in your editor.

Deliverable: `design/design-brief.md` containing:

- ASCII wireframes OR Mermaid diagrams for the board view, the card detail / comment view, and the notifications feed
- A component list with the props each component takes and what it is responsible for
- User flows for: creating a card, adding a comment that includes an @mention, and a mentioned user seeing and clicking through a notification

You do not need a design tool. A rough sketch that another engineer could implement from is the bar.

Step 3 — Plan phase (subagent)

Invoke a planning subagent (or Claude's Plan mode) to break the work into numbered implementation steps.

Deliverable: `docs/plan.md` with 7-12 numbered steps, a per-step time estimate, and dependency notes.

Tip. Ask the planning subagent to identify risky steps. Its answer often reveals the hardest bit early, before you've spent time on it.

Step 4 — Architecture phase

Use a subagent OR the superpowers plugin OR the cavern plugin to produce a technical architecture doc.

Deliverable: `docs/architecture.md` containing:

- Data model (Board, Column, Card, Comment, Notification) as a Mermaid ER diagram or ASCII
- API endpoints table with columns: method, path, purpose, auth
- State-flow narrative for: comment posted -> mention parsed -> notification created -> feed shows unread

- Two tradeoffs you explicitly considered and how you decided

Step 5 — Build the feature TDD-style

Rules:

- Write a failing test first, commit it, then implement.
- Required TDD targets: mention parsing (regex + user lookup), notification creation, and board/column/card CRUD authorization (I can only mutate my own boards).
- All unit and integration tests must pass at the end.
- Commit messages should show the tests-first pattern. For example, `test: failing test for mention parser` followed by `feat: implement mention parser`.

Gotcha. If Claude offers to write the tests and the implementation together, stop it. Ask for the failing test first, run it and see it fail, then ask for the implementation. Skipping the "see it fail" step is the single most common way TDD becomes theater.

Step 6 — HTML workflow report

Deliverable: `report/workflow-report.html`. A self-contained HTML file with inline CSS and no external assets. Required sections:

- 1 Agents and plugins used, and why you chose each.
- 2 Links to `docs/plan.md` and `docs/architecture.md`.
- 3 Test results — a pass/fail table plus a screenshot of your terminal after running the test command for your stack.
- 4 Git log excerpt for this session.
- 5 Screenshots of the built UI: board view, comment with a mention, notifications feed.
- 6 Reflection — 3-4 sentences on what surprised you and what you would change.

See `examples/sample-html-report.html` for a reference layout.

Deliverables Checklist

The same list lives in `SUBMISSION_CHECKLIST.md` in checkbox form.

- Git repo initialized and committed after each phase.
- Both `CLAUDE.md` files at both levels (user + project), manual and `/init` versions.
- `design/design-brief.md`.
- `docs/plan.md`.
- `docs/architecture.md`.
- Feature implemented, all tests passing.
- `report/workflow-report.html`.
- Git log shows tests-first commits.
- Repo pushed to a remote; share the URL.

Tips

How to invoke `/init`

In Claude Code, type `/init` in the chat. Claude scans the repo and drafts a project `CLAUDE.md`. Review the draft line by line — do not accept it wholesale.

How to run a subagent

In Claude Code, use the Task tool with `subagent_type: "general-purpose"` for open-ended research and analysis tasks, or invoke Plan mode for planning work.

How to use Plan mode

Type `/plan` (or invoke the Plan agent explicitly). Plan mode returns a step-by-step plan without executing any code, which makes it well-suited to Step 3.

How to use a plugin

If the superpowers or cavern plugin is installed, invoke it via `/superpowers` or `/cavern` (or the plugin's documented entry command). The plugin's skill will guide the architecture phase.

Grading Criteria (short version)

The full breakdown is in `RUBRIC.md`. Total: 100 points.

- CLAUDE.md quality: 15
- Design brief: 10
- Plan doc: 10
- Architecture doc: 10
- Feature correctness: 20
- TDD evidence: 15
- HTML report: 15
- Reflection / polish: 5